

OpenAAC: An Open, Offline, Multilingual Communication Aid Built on Arduino UNO Q

Arduino® UNO Q Prove it ! Hackathon

Challenge theme *Human–Machine Harmony: Assistive technologies for neurodiverse users*

Author *Eleftherios Batzolis*

Team *SystemsGenesys*

Email *e.batzolis@gmail.com*

Location *Greece*

Project repository *<https://github.com/Lefteris-B/OpenAAC>*

1. Problem statement and solution overview

1.1 The problem

Approximately 70 million people worldwide need Augmentative and Alternative Communication (AAC), devices that help people speak when speech is unreliable, costly, or impossible. The population includes people with cerebral palsy, autism, post-stroke aphasia, ALS, and locked-in syndrome. This population is poorly served on three axes:

- **Cost.** Commercial AAC devices (Tobii Dynavox, PRC-Salttillo, Smartbox) cost €5,000–€15,000. Insurance coverage outside high-income countries is sparse; coverage for "speaking-sometimes" autistic users is sparse everywhere.
- **Language.** Major commercial AAC ships with English, Spanish, and a small set of Western European languages. Greek, Arabic, Ukrainian, Romanian, and most languages of the Middle East and Sub-Saharan Africa are functionally unserved. A South African study found 54% of non-English-background AAC users had no access to their home language in any modality.
- **Connectivity assumptions.** App-based AAC increasingly assumes a smartphone, an iPad, and reliable internet. None of those hold for users in rural areas, hospitals, conflict zones, or care homes that ban personal devices.

The technical conservatism of commercial AAC compounds the gap: most devices still use 1990s-era n-gram word prediction, while the AAC research community has shown that BERT-class encoder models substantially outperform n-grams on next-pictogram and next-word prediction (PictoBERT, Pereira et al. 2022; PrAACT 2023; RoBERTa-AAC, Springer HCI 2023). These advances are not yet in cheap, repairable, multilingual hardware. No commercial vendor has assembled them. The design space is open, and the Arduino UNO Q, with its dual-brain architecture combining a Linux-capable AI processor with a real-time microcontroller, is uniquely positioned to fill it.

1.2 What we propose

OpenAAC is a battery-powered, offline communication device built around the Arduino UNO Q. The Qualcomm Dragonwing QRB2210 processor runs a custom distilled BERT-class language model (~15M parameters, distilled from a multilingual teacher) natively under Debian Linux using ONNX Runtime, while the STM32U585 microcontroller handles real-time input scanning, e-paper display refresh, and battery management telemetry. The device displays an ARASAAC pictogram grid on a 4.2" e-paper screen, accepts input via tactile buttons or a single mechanical switch, and speaks user-constructed sentences through eSpeak-NG text-to-speech. A battery management subsystem with fuel-gauging, USB-PD charging, and thermal monitoring safeguards a 2× 18650 Li-ion pack.

The device is fully offline, no cloud, no microphone, no Wi-Fi during normal use. Personalisation (fine-tuning the language model on a user's own writing) happens off-device via an opt-in companion application; trained weights sync over Wi-Fi once and the radio returns to off. All hardware design, firmware, model architecture, and corpus construction methodology will be released under permissive open-source licenses.

Scope, committed deliverables, stretch goals and future directions. The MVP is committed to one language: **English**, fully working and fully evaluated. The system is built on a multilingual foundation (multilingual DistilBERT, ARASAAC pictograms, eSpeak-NG, swappable vocabulary lists) so that adding languages is incremental work rather than architecture work; **Greek is tracked as a stretch goal**, because I am based in Greece and a usable Greek device would directly serve a population that no commercial AAC vendor reaches today. **Truly multilingual deployment**, Arabic with right-to-left rendering, Ukrainian, Romanian, and other underserved languages, is framed honestly as a future research direction that the project's open-source release is designed to enable.

Target unit cost: **under €200** in single-unit BOM. Target weight: under 400 g. Target battery life: 1.5–2.5 days of typical use on a single charge.

1.3 Why this is a "Human–Machine Harmony" project

The Arduino UNO Q Community Challenge's "Human–Machine Harmony" theme calls for assistive technologies for neurodiverse users. OpenAAC answers that call directly:

- **Neurodiverse users are the primary audience.** Adult autistic part-time AAC users are often abandoned by commercial AAC because the field assumes a child-coded, full-time-AAC user base. Post-stroke aphasia users need adult-vocabulary text-and-pictogram input that is faster than navigating a dense grid of unfamiliar symbols. These are the two populations the English MVP serves.
- **The AI at the edge is the enabler.** The Dragonwing QRB2210's quad-core Cortex-A53 at 2 GHz runs a custom distilled BERT-class language model that predicts the user's next pictogram, reducing the number of button presses to construct a sentence by roughly 40–60% compared to no prediction. This on-device intelligence is what makes the device practical, without it, grid-based AAC is too slow for conversational use.
- **The dual-brain architecture is the differentiator.** The STM32U585's real-time responsiveness ensures that button debouncing and switch scanning remain deterministic and latency-free even while the QRB2210 is running BERT inference. This split is not a

gimmick , it directly addresses documented AAC user complaints about input lag and missed key presses.

1.4 Why the Arduino UNO Q is the right platform

The UNO Q's dual-brain design is an almost ideal fit for this application:

The QRB2210 (Linux MPU) handles the AI-heavy work. With 2 GB of LPDDR4 RAM and a quad-core Cortex-A53 at 2 GHz, the QRB2210 runs a custom distilled BERT model (~15M parameters, ~15 MB in INT8) natively under Debian Linux using ONNX Runtime. While the Cortex-A53 is not fast enough to run a full 66M-parameter DistilBERT at interactive speeds (which would take 1–3 seconds per inference on this hardware), a purpose-built smaller model , 6 transformer layers, 384 hidden dimensions, 6 attention heads , achieves sub-500 ms inference while retaining the accuracy benefits of transformer-based prediction over n-gram baselines. This is a deliberate middle ground: the UNO Q has enough RAM and compute to avoid the extreme MCUBERT-class compression required on bare microcontrollers (which sacrifices accuracy), while the AAC task's 1000-class output head does not require full DistilBERT capacity. eSpeak-NG text-to-speech runs natively in Linux, producing audio output without any porting effort. Python-based personalisation scripts run directly on the board during the one-time weight-sync step.

The STM32U585 (real-time MCU) handles the safety-critical and latency-sensitive work. Button debouncing, switch-scanning state machines, e-paper SPI refresh commands, and battery telemetry (via I²C to the fuel gauge IC) all run on the MCU's Zephyr RTOS with deterministic timing. This means the AAC input system never blocks on Linux scheduler latency , a critical requirement for users with motor impairments who depend on precise scan timing.

The Arduino Bridge library connects both sides. The QRB2210 sends prediction results to the STM32U585 for display update; the STM32U585 sends button events to the QRB2210 for BERT inference. The RPC-based Bridge library handles this transparently.

The UNO form factor and ecosystem simplify hardware development. Arduino-compatible headers, Qwiic connector for expansion, 16 GB eMMC for model and pictogram storage, and built-in Wi-Fi 5 for the one-time personalisation sync , all features that would otherwise require custom PCB design.

Cost. The UNO Q 2 GB retails for approximately €48, providing compute capabilities that would otherwise require a custom PCB with a QRB2210, DDR, eMMC, and power management , easily a €100+ design at prototype volumes.

2. Hardware design

The hardware consists of the Arduino UNO Q as the compute core, surrounded by four peripheral subsystems connected via the UNO headers, Qwiic, and GPIO. Figure 1 (*to be inserted*) shows the block-level architecture.

2.1 Compute core

Component	Role	Approx. cost
Arduino UNO Q (2 GB variant)	Main compute platform. QRB2210 MPU runs BERT inference, TTS, and UI logic under Debian Linux. STM32U585 MCU runs real-time input handling, e-paper SPI, and battery telemetry on Zephyr RTOS. 16 GB eMMC stores model, pictograms, eSpeak voices, and OS. Built-in Wi-Fi 5 for one-time personalisation sync.	€48

The 2 GB variant is chosen over the 4 GB to minimise cost and power draw. The custom BERT model in INT8 quantisation requires approximately 15 MB of working memory plus ~50 MB for runtime buffers; even with the Debian OS overhead (~300–400 MB resident), 2 GB provides ample headroom. The 16 GB eMMC holds the Debian OS (~3 GB), the model (~15 MB), the ARASAAC pictogram set (~50 MB for 1000 pre-rendered 1-bit images), eSpeak voice data (~20 MB), and application code with room to spare.

2.2 Battery management subsystem

Component	Role	Approx. cost
MAX17320	2S Li-ion fuel gauge with internal cell-balancing FETs and ModelGauge m5 EZ algorithm	€6
MP2731	USB-PD-aware buck-boost charger; negotiates input voltage, manages charge profile	€3
TPS62A02	Buck regulator producing 5 V rail for UNO Q (the board requires 5 V / 3 A for full performance)	€2
INA226	Bidirectional current/voltage monitor on battery output	€2
External N-FET cell protection circuit	Hardware-level overcurrent and short-circuit safety	€2
2× NTC thermistors	Battery and charger thermal monitoring	€0.50

The STM32U585 on the UNO Q reads battery telemetry from the MAX17320 and INA226 over I²C, monitors NTC temperatures via ADC, and controls the MP2731 charge profile, all running on the MCU's Zephyr side with deterministic timing. The MCU can signal the Linux side to enter suspend when battery is low, and wake it on button press. This approach leverages the UNO Q's built-in dual-brain architecture: the STM32U585's low-power stop mode (~2.5 µA) keeps battery monitoring alive even when the Linux side is suspended.

2.3 Display and audio subsystem

Component	Role	Approx. cost
Waveshare 4.2" e-paper module, 400×300, B/W	Main display; zero current when not refreshing; readable in sunlight	€23
USB audio adapter (USB-C to 3.5 mm DAC)	Routes eSpeak-NG audio from QRB2210's USB subsystem to the speaker amplifier	€3
PAM8403 Class-D audio amplifier module	Amplifies line-level audio from USB DAC to drive speaker	€1.50
Small 4 Ω, 3 W speaker	Front-firing for clarity	€4

E-paper is essential for battery life , its zero-idle-current property dominates in a device that spends most of its time displaying a static grid. The STM32U585 handles the SPI refresh commands to the e-paper controller with precise timing, receiving display update commands (pictogram IDs and grid positions, not raw framebuffer data) from the Linux side via the Bridge RPC , this keeps Bridge traffic to a few hundred bytes per update rather than a full 15 KB framebuffer.

Audio routing note. The QRB2210's native audio endpoints are on the JMISC bottom-side connector, which requires either the Arduino UNO Media Carrier (announced March 2026, provides 3.5 mm jacks) or a custom JMISC breakout board. For the MVP, we use a USB audio adapter connected to the QRB2210's USB-C subsystem via an internal hub, which is simpler and works immediately with standard ALSA drivers. If the UNO Media Carrier becomes available during the build phase, the design can migrate to the JMISC analog audio path for lower component count.

2.4 Input subsystem

Component	Role	Approx. cost
11 tactile momentary switches	5 prediction-slot buttons, 5-key d-pad with OK, 3 function buttons (home, back, speak)	€3
3.5 mm mono jack for switch scanning	Single-switch input for users with severe motor impairment	€1
Power switch	Primary on/off	€0.50
Status LED	Charging / system status	€0.30

All buttons connect to the STM32U585's GPIO pins via the UNO headers. The MCU debounces inputs and runs the switch-scanning state machine in Zephyr, then sends events to the Linux side via the Bridge RPC. This guarantees deterministic input timing regardless of Linux scheduler load , critical for single-switch scanning, where a 50 ms timing error means a missed selection.

2.5 Power subsystem

Component	Role	Approx. cost
2× 18650 Li-ion cells (used, valued at market price)	2S configuration, ~7.4 V nominal, ~3000 mAh per cell	€10
2-cell 18650 holder	Mechanical mounting	€2
USB-C connector with PD support, ESD protection, TVS array	Charging input	€2

The UNO Q requires 5 V / 3 A for full performance. The TPS62A02 buck regulator steps down from the 2S Li-ion pack (~6.0–8.4 V) to a stable 5 V rail. Independent power measurements (Jeff Geerling, October 2025) show the UNO Q draws approximately 0.5 W at idle and 1.9–2.5 W under sustained compute load. AAC use is bursty , brief inference bursts (~0.5 s at ~2 W) separated by seconds-to-minutes of idle waiting for user input (~0.5 W) , so average active-use power is estimated at 0.7–1.0 W. With a 22 Wh battery pack and 8 hours active / 16 hours idle per day (0.5 W, or lower if Linux suspend-to-RAM is achievable on the QRB2210), the device targets 1.5–2.5 days per charge depending on usage intensity and suspend effectiveness. Linux suspend on mobile Qualcomm SoCs is architecturally supported but not yet verified on the UNO Q's Debian distribution; this is tracked as a build-phase validation task.

2.6 Mechanical / enclosure

Component	Role	Approx. cost
Custom 3D-printed enclosure (PETG)	Front face with screen window and button cutouts; UNO Q mounting, rear-mounted battery compartment; emergency paper card slot	€5
Gaskets and port covers	IP54-target ingress protection	€2
Hardware (screws, threaded inserts)		€2

The UNO form factor simplifies mechanical design , the board's standard mounting holes and header positions are well-documented. The enclosure design will be released as printable STL files.

2.7 BOM summary

Subsystem	Cost
Compute (Arduino UNO Q 2 GB)	€48
Battery management (fuel gauge + charger + protection + monitoring)	€15.50
Display + audio	€31.50
Inputs	€4.80
Power (cells, holder, USB-C, buck regulator)	€14
Enclosure and mechanicals	€9
Custom carrier board / shield (PCB fabrication, JLCPCB-class)	€20
Reserve for component substitution	€20
Total	~€161 / ~\$175

The custom carrier board is an Arduino shield that hosts the BMS ICs, e-paper connector, audio amplifier, button connectors, and 3.5 mm jack , stacking directly onto the UNO Q's headers. This avoids a custom main PCB entirely, leveraging the UNO ecosystem's modularity.

3. Software design

3.1 System architecture

The software architecture exploits the UNO Q's dual-brain split: the QRB2210 runs the AI and application logic under Debian Linux, while the STM32U585 runs the real-time I/O firmware on Zephyr RTOS. The Arduino Bridge library provides the RPC link between the two.

On the QRB2210 (Linux / Python):

- **BERT inference engine.** Custom distilled BERT model (6 layers, 384 hidden, 6 heads, ~15M parameters) loaded via ONNX Runtime with INT8 quantisation. On each user selection, runs forward inference and returns top-5 next-pictogram predictions over the 1000-class vocabulary. Inference time target: <500 ms on the Cortex-A53 cluster. Note: full DistilBERT-base (66M parameters) was evaluated and rejected , it runs at 1–3 seconds per inference on the A53, which is too slow for interactive AAC use.

- **UI manager.** Python application that maintains screen state, determines which pictograms and text to display at which grid positions, and sends compact display-update commands (pictogram IDs, positions, text strings) to the MCU for e-paper refresh. The MCU composites the display locally from pre-stored bitmap tiles, avoiding the need to transfer full 15 KB framebuffers over the Bridge RPC.
- **Sentence buffer.** Manages the user's in-progress utterance , pictogram IDs and assembled text.
- **TTS engine.** eSpeak-NG (GPL v3) runs natively on Linux; generates WAV audio and routes to the USB audio adapter via ALSA.
- **Persistence layer.** Reads/writes user vocabulary, recently-used cache, and personalisation weights from eMMC.
- **Personalisation.** On-device Python scripts accept user writing samples and fine-tune the final 2 layers and output head. Runs during a one-time setup session.
- **Power management.** Receives battery state from the MCU via Bridge; manages Linux suspend/resume based on idle timeout.

On the STM32U585 (Zephyr RTOS / Arduino sketch):

- **Input handler.** Debounces buttons on GPIO; runs switch-scanning state machine when 3.5 mm jack is connected; sends events to Linux via Bridge RPC.
- **E-paper driver.** Stores pre-rendered 1-bit pictogram bitmaps in MCU-accessible flash; receives display update commands (pictogram IDs and grid positions) from Linux via Bridge RPC; composites the screen locally and drives SPI partial and full refreshes with precise timing; manages ghosting accumulation counter.
- **Battery telemetry.** Reads MAX17320 (cell voltage, SOC) and INA226 (current) over I²C; reads NTC temperatures via ADC; controls MP2731 charge profile; reports state to Linux via Bridge.
- **Wake/sleep control.** Detects button press or switch-jack activity; wakes Linux from suspend; triggers suspend on idle timeout.

3.2 Language model

Architecture. A custom 6-layer transformer encoder distilled from `distilbert-base-multilingual-cased` (Apache 2.0). Hidden size 384, 6 attention heads, FFN intermediate 1536, ~15M parameters before quantisation, ~15 MB after INT8 post-training quantisation. The output head is a 1000-way classifier over the pictogram vocabulary (700 fixed core + 300 user-customisable slots). Sequence length 32 tokens , AAC utterances are short and this limits compute.

Why not full DistilBERT? The full 66M-parameter DistilBERT-base (768 hidden, 12 heads) takes 1–3 seconds per inference on the QRB2210's Cortex-A53 cores , too slow for interactive use. The custom smaller architecture targets sub-500 ms inference while still substantially outperforming n-gram baselines. This is a meaningful advantage over the pure-MCU approach (which requires compressing BERT to ~3 MB / 6.5M parameters with significant accuracy loss); the UNO Q's Linux environment allows a model 2–3× larger than what fits on a microcontroller, with FP32 or INT8 inference rather than aggressive bit-packing.

Training pipeline (off-device, before build phase):

1. **Teacher model.** `distilbert-base-multilingual-cased` (Apache 2.0, 66M parameters), used as the distillation teacher. The multilingual teacher preserves the option to add Greek and other languages without retraining from scratch.
2. **Student distillation.** The 6-layer, 384-hidden student is trained to match the teacher's logits and embeddings on English Wikipedia and CHILDES corpora. Loss: KL divergence on logits + cosine embedding loss. Estimated training time on an NVIDIA RTX 2080 Super: ~12 hours.
3. **Vocabulary reduction.** Following the Geotrend "Load What You Need" methodology (arXiv:2010.05609), reduce the 110K-token multilingual vocabulary to ~6K English-only WordPiece tokens for the MVP, collapsing the embedding table. The same procedure applied to English+Greek tokens (~8K) produces the stretch-goal model with no architectural change.
4. **AAC fine-tuning.** Replace the MLM head with the 1000-way pictogram classification head; fine-tune on the AAC-specific corpus. Estimated time: ~4 hours.
5. **Quantisation.** Post-training INT8 quantisation via ONNX Runtime; verify ≤ 3 percentage point drop in top-5 accuracy. **Latency validation:** first on-device inference benchmark must be run in week 2 of the build phase; if latency exceeds 500 ms, further layer or dimension pruning will be applied.

Corpus construction. AAC utterances have a register that does not appear at scale on the open web (telegraphic, short, function-word-reduced). The English corpus is constructed from four sources:

- **CHILDES** child-directed and child-produced speech (open licence; closest natural register match)
- **OPUS OpenSubtitles** filtered to utterances of 3–8 words with profanity and adult-content removal
- **Synthetic AAC text** generated by a teacher LLM with controlled prompts for AAC register and topic coverage; ~50K utterances
- **ARASAAC pictogram-aligned sentences** where available, providing direct supervision for the pictogram-prediction task

Target: 200K–500K English utterances. Greek corpus follows the same procedure if pursued as a stretch.

3.3 Pictogram vocabulary (1000 classes)

A four-tier structure derived from the AAC core/fringe literature (Banajee 2003, Beukelman 1984, Stuart-Beukelman 1997, Boenisch & Soto 2015):

Tier	Size	Description
1. Universal core	~50	Highest-frequency function words and pronouns (I, you, want, more, no, yes, help). Fixed anchor positions on every UI page.
2. Adult-conversational	~200	High-frequency adult vocabulary covering ~78% of typical adult speech.
3. Domain-essential	~250	Emotional, medical, identity, and relationship vocabulary.
4. Generic-fringe	~200	Categorisable vocabulary (food, places, time, weather) accessed via category browsing or prediction.
5. User-customisable slots	300	Empty by default; filled via companion app with personal nouns (family names, pet names, doctor's name).

Pictograms are sourced from ARASAAC (CC BY-NC-SA licence), pre-rendered to 50×50 1-bit dithered images for e-paper and stored on the UNO Q's eMMC.

3.4 User experience

The device runs a fixed-grid UI with a separate prediction strip. This design follows a documented AAC user-research finding: AAC users rely heavily on motor-planning consistency (LAMP-style fixed positions), and reordering the main grid by prediction destroys muscle memory.

Three input modalities:

- **Pictogram grid mode (primary).** 3×6 grid with leftmost anchor column fixed across pages; 5-cell prediction strip above; sentence display at top; status bar at bottom.
- **QWERTY mode (secondary).** For literate users; same prediction strip showing word completions; d-pad navigation.
- **Single-switch scanning (tertiary).** Row-column scanning with configurable scan rate; BERT-driven prediction prioritisation is most valuable here, where wrong predictions cost seconds of dwell time.

Output: e-paper sentence display + manual-trigger eSpeak-NG TTS through the speaker. The device never speaks unbidden, user dignity requires that the assembled sentence is finalised and committed before voicing.

A laminated paper pictogram card slides into a slot in the enclosure, providing a low-tech backup for emergency use and battery-dead states. This addresses documented AAC user failure modes (Boesch et al. 2022, Hurricane Harvey communication study).

3.5 Software development workflow

Development uses Arduino App Lab, which is pre-installed on the UNO Q. The Python application running on the QRB2210 and the Arduino/Zephyr sketch running on the STM32U585 are developed in a single unified project. The Bridge library handles inter-processor communication. This substantially reduces development complexity compared to a custom dual-MCU design where two separate toolchains, two flashing procedures, and a custom communication protocol would all need to be implemented and debugged.

Bridge maturity note. The Arduino Bridge/Router RPC library is relatively new (launched October 2025) and community documentation is still sparse. Early adopter reports on the Arduino Forum suggest the core RPC mechanism works reliably for simple calls but the ecosystem is still maturing. Our build-phase schedule includes a dedicated Bridge validation task in week 1: if RPC round-trip time exceeds 50 ms or message size limits prevent our display-update protocol, we will fall back to direct UART communication between the two processors using a custom lightweight protocol.

3.6 Software licences and acknowledgements

- Debian Linux (various open-source licences)
- Arduino App Lab and Bridge (Arduino licence)
- DistilmBERT (Apache 2.0)
- ONNX Runtime (MIT)
- ARASAAC pictograms (CC BY-NC-SA 4.0)

- eSpeak-NG (GPL v3); application code interfacing with eSpeak-NG is released under GPL v3
 - Hugging Face transformers (Apache 2.0) for training infrastructure
-

4. Evaluation plan

The project will measure five quantitative metrics:

1. **Top-5 next-pictogram prediction accuracy** on held-out AAC sentences in English. Target: $\geq 42\%$ (PictoBERT reported 48% with a larger model; our smaller distilled model should achieve at least 42%, which is above the $\sim 40\%$ threshold below which prediction stops being useful). *Stretch*: same metric for Greek.
2. **Words-per-minute** on a fixed simulated test set of representative phrases ("I want water", "I am in pain", "thank you for visiting"), measured for each input modality. Target: ≥ 15 WPM in grid mode. (Conservative relative to RoBERTa-AAC's 25.75 WPM because e-paper refresh latency adds ~ 300 ms per selection.)
3. **BERT inference latency** measured on-device. Target: < 500 ms per forward pass. (Based on ARM Cortex-A53 performance estimates for our 15M-parameter INT8 model; validated in build-phase week 2.)
4. **End-to-end battery life** under a realistic duty cycle (8 active hours, 16 idle/suspend hours per day). Target: ≥ 1.5 days. (Based on measured UNO Q idle power of 0.5 W and estimated average active power of 0.7–1.0 W.)
5. **E-paper refresh latency** for partial and full refresh.

Comparative baselines: 5-gram language model (no neural) and unigram frequency. Including these baselines makes the contribution of the BERT-class model legible and provides a fallback.

5. Schedule

Phase 1: Foundation (weeks 1–2)

- **English vocabulary derivation.** 700-word core list from AAC literature; ARASAAC label coverage audit; pictogram pre-rendering at 50×50 1-bit.
- **Corpus construction.** CHILDES English; OPUS filtering; synthetic AAC text generation.
- **Model distillation and fine-tuning.** 6-layer student distillation from DistilmBERT teacher; AAC fine-tune; INT8 quantisation; first accuracy evaluation.
- **Hardware procurement.** Arduino UNO Q 2 GB, 4.2" e-paper, MAX17320 and MP2731 evaluation boards, USB audio adapter, 18650 cells.
- **Bridge validation (week 1, critical).** Measure Bridge RPC round-trip time and maximum message size; verify display-update command protocol is feasible; if not, begin UART fallback implementation.
- **On-device latency validation (week 2, critical).** Run first BERT inference on UNO Q; measure latency. If > 500 ms, immediately begin model pruning (reduce to 4 layers / 256 hidden).

Phase 2: Integration (weeks 3–4)

- **Linux-side application.** Python BERT inference loop; UI state management; display-update command protocol; eSpeak-NG integration via USB audio adapter / ALSA; Bridge RPC client.
- **MCU-side firmware.** Button debouncing; e-paper SPI driver with local bitmap compositing; battery telemetry; Bridge RPC server (or UART fallback).
- **First end-to-end demo.** Button press → BERT prediction → e-paper update → TTS output.
- **Carrier shield design.** KiCad schematic and layout for the BMS + peripherals shield; PCB fabrication ordered.

Phase 3: Polish and evaluation (weeks 5–6)

- **Full UX integration.** Grid mode, QWERTY mode, switch scanning.
- **Battery life measurement.** INA226-based power profiling; Linux suspend/resume tuning.
- **Carrier shield assembly and integration.** Move from breadboard BMS to shield.
- **Evaluation suite.** All five metrics measured and documented.
- **Enclosure.** 3D-printed PETG enclosure with button cutouts and screen window.
- *Stretch:* Greek vocabulary swap, Greek eSpeak voice, Greek model variant.

Phase 4: Documentation and submission

- **DesignSpark article.** Full write-up with methodology, results, schematics, and source code links.
- **Open-source release.** All design files, firmware, model weights, training scripts, and vocabulary derivation methodology published to GitHub.
- **Demonstration video.**

Critical-path risks and mitigations

Risk	Mitigation
BERT inference latency >500 ms on QRB2210	Reduce model dimensions further (4 layers, 256 hidden); apply ONNX graph optimisation; explore Qualcomm AI Engine / Hexagon DSP offload if available in the UNO Q's Linux BSP
Linux suspend-to-RAM not functional on QRB2210 Debian	Fall back to aggressive application-level idle (stop inference loop, release ONNX session memory, blank display); accept ~0.5 W idle instead of target <0.1 W suspended
Bridge RPC latency or reliability issues	The Bridge library is new and documentation is sparse. Mitigation: validate Bridge round-trip time in week 1; if >50 ms per call, switch to direct UART or SPI communication between the two processors, bypassing the RPC layer
Audio via USB adapter introduces latency or compatibility issues	Fall back to PWM audio from STM32U585 GPIO (lower quality but functional); or if UNO Media Carrier ships, migrate to JMISC analog audio
QRB2210 thermal throttling under sustained inference	AAC inference is bursty (sub-second bursts with multi-second gaps), so sustained thermal load is unlikely; if observed, add passive heatsink to SoC or reduce model batch size
Battery life under 1 day	Larger battery pack (3S configuration or higher-capacity 18650 cells); or accept shorter battery life with fast USB-PD charging

Risk	Mitigation
Greek stretch goal not achievable	English MVP ships as the committed deliverable; Greek documented as first community contribution

6. Sustainability and social impact

Impact

- **Social.** The device targets populations whose communication needs are not met by the commercial market, adult AAC users, neurodiverse users, and speakers of underserved languages. The cost gap between OpenAAC (~€161) and commercial AAC (€5,000–€15,000) is the project's primary social-impact lever.
- **Accessibility.** Three input modalities (grid, QWERTY, single-switch scanning) serve users across the motor-ability spectrum. Offline operation serves users in environments without reliable connectivity.

Sustainability

- **Repairability.** Modular design: the UNO Q, e-paper, speaker, and battery are all independently replaceable. Through-hole 18650 holders allow cell replacement without soldering. The carrier shield uses standard through-hole and large SMD components.
 - **Reused components.** 18650 cells sourced from used laptop battery packs (a substantial waste stream); cell selection and matching documented.
 - **Material choice.** Enclosure printed from PETG (recyclable); avoidance of mixed-material assemblies.
 - **Open hardware and software.** All design files released under permissive licences (Apache 2.0, CC BY-NC-SA, GPL v3 as appropriate). The project is structured so that adding a new language is a documented procedure with a script-driven pipeline, enabling community contributions for languages the original team cannot test.
 - **Repair documentation.** A repair manual will be included, written at a level that allows a non-engineer to replace a cell, swap the e-paper, or reflash firmware.
-

7. Innovation and originality

OpenAAC integrates several published research lines that have not previously been combined into a working device:

1. **BERT-for-AAC prediction** (PictoBERT, PrAACT, RoBERTa-AAC), applying transformer-based language models to next-pictogram prediction.
2. **Multilingual corpus construction for AAC** (BERTimbau-AAC), building AAC-register training data from mixed sources for underserved languages.
3. **Right-sized on-device edge AI for assistive technology**, running a custom distilled BERT model sized specifically for the QRB2210's compute budget: large enough to outperform n-gram baselines substantially, small enough for sub-500 ms interactive inference, and hosted entirely on-device for privacy and offline operation.

4. **The ARASAAC open pictogram set** , the world's largest open-licence symbol library, supporting 40+ languages.
5. **The UNO Q's dual-brain architecture as an assistive-technology platform** , using the QRB2210 for AI and the STM32U585 for deterministic real-time I/O, demonstrating that the board's design is not just for robotics and vision but also for human communication.

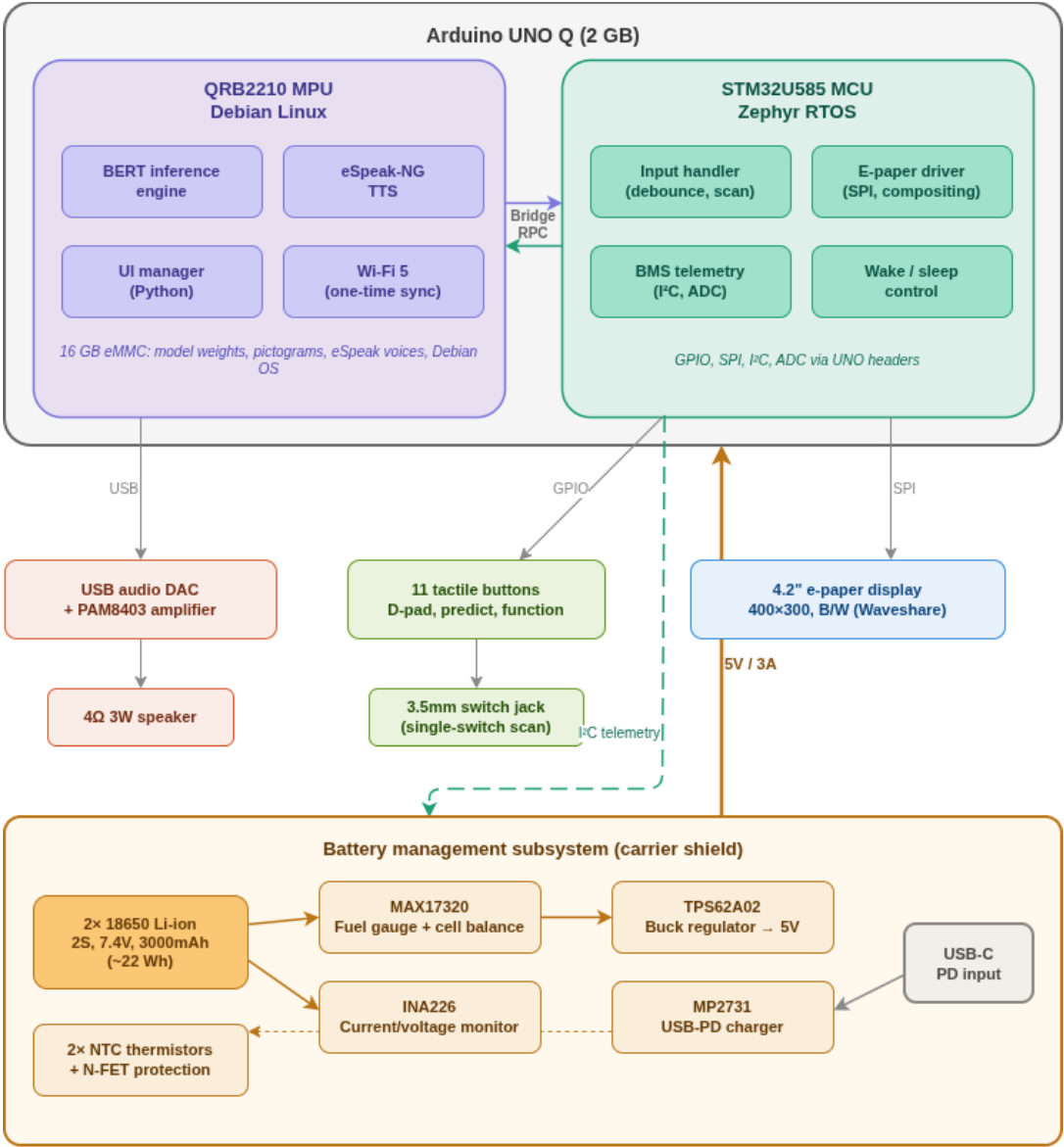
The UNO Q is a particularly strong fit because the AAC use case genuinely needs both brains: the AI processor for language modelling, and the real-time controller for the precise, deterministic input timing that motor-impaired users depend on. This is not a project that could run on either half alone.

To our knowledge, no existing AAC device , commercial or research , combines on-device transformer inference with offline operation, hobbyist-affordable BOM, and an architecture designed for extensibility to underserved languages. The closest commercial comparables are 30–60× more expensive and serve a small set of high-resource languages.

8. Judging criteria alignment

Criterion	How OpenAAC addresses it
Innovation	First AAC device combining on-device transformer inference with the UNO Q's dual-brain architecture; purpose-built distilled BERT model sized for the A53's compute budget; novel application of edge AI to assistive communication; multilingual-by-design corpus and model pipeline.
Feasibility	All components are commercially available; software stack runs on standard Debian Linux with well-supported libraries (ONNX Runtime, eSpeak-NG); BOM is under €200; schedule includes explicit risk mitigations.
Demonstration	Five quantitative evaluation metrics with comparative baselines; end-to-end demonstration video showing button-to-speech workflow; open-source repository with reproducible build instructions.
Impact	Serves 70 million people needing AAC worldwide; targets adult neurodiverse users specifically underserved by commercial AAC; 30–60× cost reduction vs. commercial devices; open-source release enables community-driven language expansion.

9. System block diagram



Legend

- Linux MPU domain
- Real-time MCU domain
- Power / BMS
- Display
- Input
- Audio

Figure 1: System Block diagram

10. Software architecture and data flow

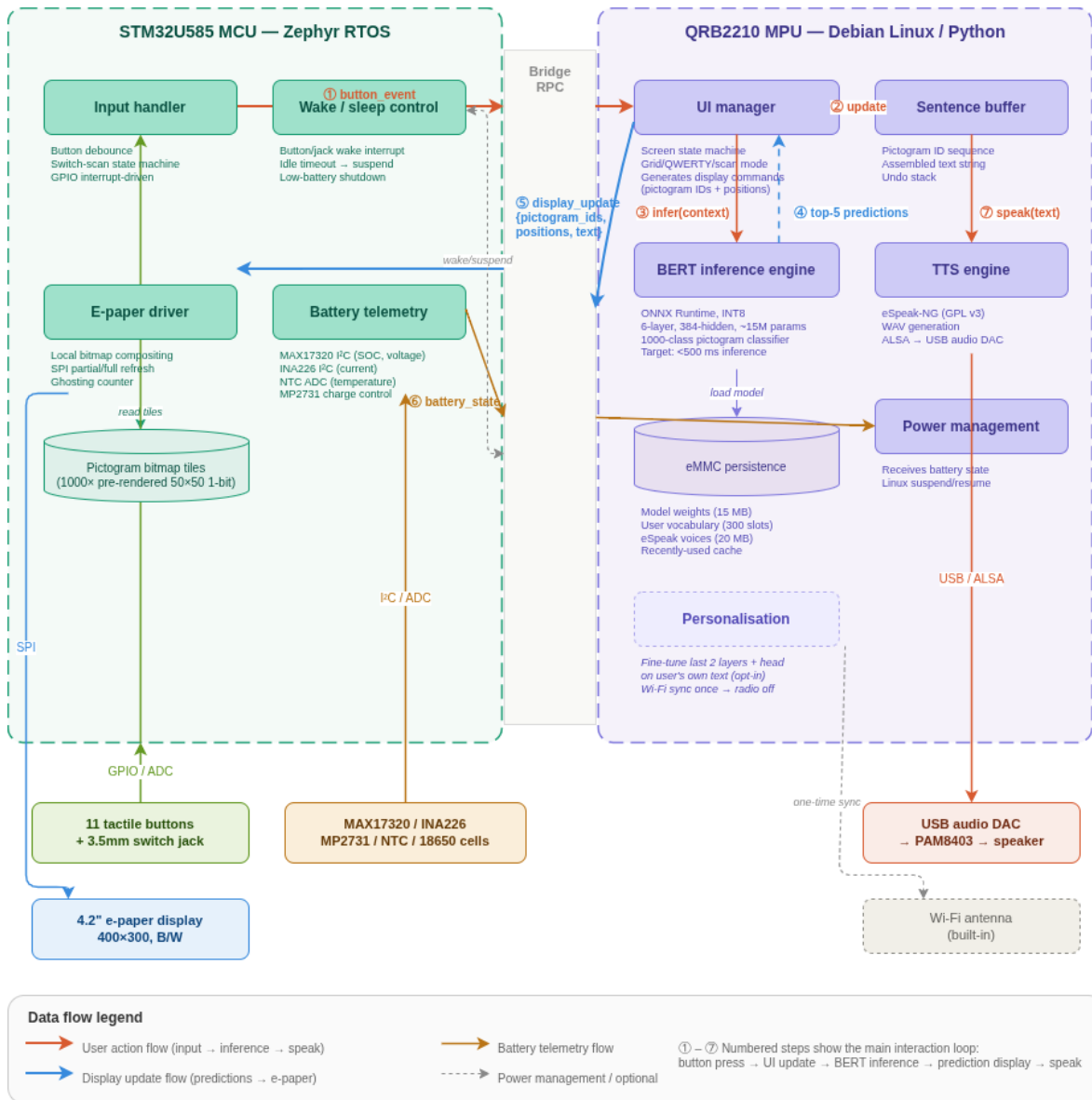


Figure 2: Software architecture and data flow diagram

11. Acknowledgements and references (selected)

- Pereira, J. et al. (2022). *PictoBERT: Transformers for next-pictogram prediction*. Expert Systems with Applications.
- Banajee, M., DiCarlo, C., & Stricklin, S. (2003). *Core vocabulary determination for toddlers*. AAC.
- Stuart, S., Beukelman, D., King, J. (1997). *Vocabulary use during extended conversations by older adults*. AAC.
- Boesch, M. et al. (2022). *Communication during emergency response: AAC users in Hurricane Harvey*.
- Donaldson et al. (2021). *"Everyone Deserves AAC": Perspectives on AAC across the spectrum of communication ability*. ASHA Perspectives.
- ARASAAC Pictogram Symbol Set, Government of Aragón, CC BY-NC-SA 4.0, <https://arasaac.org>

- DistilmBERT, Sanh et al., Apache 2.0, <https://huggingface.co/distilbert/distilbert-base-multilingual-cased>
 - eSpeak-NG, GPL v3, <https://github.com/espeak-ng/espeak-ng>
 - Geotrend "Load What You Need" multilingual extraction methodology, arXiv:2010.05609
 - Arduino UNO Q documentation, <https://docs.arduino.cc/hardware/uno-q>
-